



Lab Course:
Spacecraft Computer, Software and Data
Final Report

Andrei Arteev

Alisher Orynbek

Alberto Sánchez Calvo

Frederik Alexander Simon

Angela Ghurburrun

Keval Satish Mohite

Konstantin Hesselmann

July 2026

Contents

1	Introduction	3
1.1	Mission Overview	3
1.2	Scope of this Report	3
1.3	Team and Responsibilities	3
2	Machine-Learning Payload	5
2.1	What the Payload Does	5
2.2	Proof-of-Concept Scope	5
2.2.1	Minimum Demonstration	6
2.2.2	Failure Conditions	6
2.2.3	Why a TPU at All	6
2.3	Implementation	7
2.3.1	Model: Two Candidates, One Deployed	7
2.3.2	Training Pipeline	7
2.3.3	Coral Firmware	7
2.3.4	Wire Interface	8
2.4	Thermal Envelope	8
3	Command and Data Handling (CDH)	9
3.1	Responsibilities	9
3.2	Interfaces	9
3.3	Design Decisions	9
4	Telemetry, Tracking and Command (TT&C)	10
4.1	Subsystem Overview	10
4.1.1	Responsibilities	10
4.2	Requirements and Traceability	11
4.3	Physical and Radio Interface	12
4.4	Telemetry Generation and Scheduling	13
4.5	Protocol-v8 Telemetry Frame	13
4.6	Reliable Downlink and Uplink Protocol	15
4.6.1	Commands and Replay Protection	16
4.7	Flight Software State Machines	17
4.8	FDIR and Radio Health	17
4.9	Ground Segment	18
4.10	Link Budget, Airtime and Channel Use	18
4.11	Verification Evidence	19
4.12	Design Decisions and Known Limitations	19

5	On-Board Computer (OBC)	20
5.1	Responsibilities	20
5.2	Interfaces	20
5.3	Design Decisions	21
5.3.1	SD Card Storage	22
6	Electrical Power System (EPS)	24
6.1	Design Overview	24
6.2	Power Budget	24
6.3	Power Architecture	25
6.4	Verification	26
7	Flight Software	28
7.1	Mission Phases	28
8	Structures	29
8.1	Responsibilities	29
8.2	Interfaces	29
8.3	Design Decisions	29

Chapter 1

Introduction

1.1 Mission Overview

Lakitu is a 1U CubeSat-format payload flown aboard a stratospheric balloon, developed as a Spacecraft Computer, Software and Data (SCSD) course project at the Technical University of Munich. The platform executes an autonomous 4–6 hour flight managed by a six-phase finite state machine: *Standby*, *Launch*, *Ascent*, *Cruise*, *Descent*, and *Landing*. Its primary payload is a downward-facing camera coupled to an on-board machine-learning model that performs visual cloud-cover estimation in flight. Every captured frame is archived to an SD card and a compact cloud-fraction record is downlinked over LoRa alongside housekeeping telemetry.

1.2 Scope of this Report

This report documents the spacecraft from a Computer, Software and Data perspective, organised by subsystem: Command and Data Handling (CDH), Telemetry, Tracking and Command (TT&C), the On-Board Computer (OBC), Electrical Power System (EPS), Flight Software (FSW) and its finite state machine, the on-board Machine-Learning Payload, and Structures. Each chapter covers that subsystem’s responsibilities, interfaces to the rest of the spacecraft, and key design decisions as implemented in the [scsd-lakitu-cubesat](https://github.com/k-hesselmann/scsd-lakitu-cubesat) repository (<https://github.com/k-hesselmann/scsd-lakitu-cubesat>).

1.3 Team and Responsibilities

Everyone: please check the table below is correct for your own contributions and flag anything that needs correcting.

Name	Led	Helped with
Andrei Arteev	FSW	EPS
Alisher Orynbek	OBC	–
Alberto Sánchez Calvo	TT&C	–
Frederik Alexander Simon	CDH	–
Angela Ghurburrun	Structures	–
Keval Satish Mohite	Project Management, FSW	–
Konstantin Hesselmann	EPS, Payload	OBC, Structures

Table 1.1: Team leads and supporting contributions by subsystem

Chapter 2

Machine-Learning Payload

Code, trained models, and training pipeline for this chapter live under [payload/](#) in the [scsd-lakitu-cubesat](#) repository.

2.1 What the Payload Does

Lakitu carries a small camera and a dedicated AI chip pointed straight down at the ground. Each capture cycle it takes a picture, runs it through a small neural network on board, and reduces that picture to a single number: how cloudy the sky looks, from 0% (clear) to 100% (overcast). Only that number goes down the slow radio link – the picture itself stays archived on the SD card. The camera and its AI chip (a Coral Dev Board Micro with a Google Edge TPU) form the ML payload; the STM32 OBC treats it as just another sensor polled over a serial link.

The flight exists to prove exactly two things, and nothing more:

1. **Shrink what has to be downlinked.** A raw frame is ~ 50 KB, far more than the LoRa budget can carry; reducing it to a 2-byte fraction on board is a $\sim 25,000:1$ data reduction. Raw imagery is never downlinked, only archived.
2. **Prove an Edge TPU survives and performs on a CubeSat-class platform.** A commercial, off-the-shelf AI accelerator never designed for spaceflight is asked to run continuous inference within a 1U CubeSat's power, thermal, mass, and bandwidth budget – the claim is about integration, not about the cloud model being scientifically novel.

2.2 Proof-of-Concept Scope

The PoC draws a hard boundary: the payload runs inference, produces structured output, and hands it to the OBC over UART; packetization, LoRa transmission, and ground reception belong to the OBC and TT&C subsystems. The engineering claim under test – a commercial edge accelerator performing useful on-board visual inference within a 1U CubeSat's envelope, delivering structured output into the standard telemetry stream – is treated as falsifiable: Table 2.2 lists the conditions under which it fails.

2.2.1 Minimum Demonstration

The PoC counts as proven only if all four conditions below hold together. The payload owns 1, 3, and 4 directly; 2 depends jointly on the OBC and TT&C link, so the OBC’s SD archive (keyed by the same sequence number, SEQ, as the downlink) is the fallback verification path if the downlink fails.

Criterion	What it means
Autonomous inference	Continuous output through the flight, no ground intervention, at the configured cadence.
End-to-end downlink	Fraction records received and decoded correctly on the ground via the OBC and TT&C chain.
Within envelope	Power draw inside the EPS allocation; interior above 0°C; no thermal shutdown or throttling.
Verifiable against ground truth	Downlinked/archived cloud fraction agrees with the archived source frame at a documented accuracy, post-flight.

Table 2.1: PoC minimum demonstration criteria

2.2.2 Failure Conditions

Failure mode	Diagnosis	Attribution
No inference output produced	Coral, camera, or inference pipeline did not run end-to-end	ML payload
Inference produced but not delivered to OBC	Coral output stage or UART interface failed	ML payload
Inference delivered but not received on ground	OBC packetization, TT&C transmission, or ground reception failed	OBC / TT&C / ground
Envelope exceeded (shutdown/throttle)	Power, thermal, or compute budget was misjudged	ML payload
Output disagrees with ground truth	Incorrect inference, or data corrupted in transit	ML payload / downstream

Table 2.2: PoC failure conditions and attribution

2.2.3 Why a TPU at All

The Edge TPU’s value in this PoC is sustained inference throughput inside the platform’s power envelope, not enabling a task that would otherwise be impossible on a CPU. Edge TPU inference latency for a MobileNet-class model is on the order of milliseconds, so the model runs well inside each capture cycle with margin to spare – CPU-only inference at the same cadence would draw more power per inference and erode the EPS budget instead.

2.3 Implementation

2.3.1 Model: Two Candidates, One Deployed

Two model formulations were trained on the same 95-Cloud / 38-Cloud satellite imagery dataset: a **3-class classifier** (`clear/overcast/partial`) and a **continuous regressor** producing a single sigmoid cloud-fraction output in $[0, 1]$. The regressor was selected for flight – strictly more informative at the same output cost, and it keeps the telemetry field (`fraction × 65535`) simple rather than needing a class enum. The classifier pipeline remains in the repository but is not built into the flight firmware.

Both share a MobileNet-class CNN backbone. The regressor is trained with quantization-aware training (QAT) rather than plain post-training quantization, then compiled for the Edge TPU; the deployed artifact is committed directly under `payload/coral/models/cloud_regressor_qat_int8_edgetpu.tflite` (an exception to the repo’s usual `*.tflite` ignore rule) so the firmware builds without regenerating the model.

2.3.2 Training Pipeline

Stage	Artifact / tool
Raw imagery	95-Cloud / 38-Cloud dataset (<code>payload/data/</code> , not tracked)
Class dataset build	<code>training/build_cloud_classes.py</code>
Training	<code>training/notebooks/train_cloud_regressor.ipynb</code> (Keras)
Quantization	QAT → int8 TFLite
Edge TPU compile	<code>edgetpu_compiler</code>
Flight artifact	<code>coral/models/*_edgetpu.tflite</code> (tracked)

2.3.3 Coral Firmware

The flight firmware (`payload/coral/src/cloud_regressor.cc`) is an out-of-tree `coralmicro` application (FreeRTOS on the M7 core). Per cycle it captures a 224×224 grayscale frame, invokes the int8 model on the Edge TPU via an 8 MB tensor arena in SDRAM, and sends the result to the OBC. Two bring-up findings are worth noting:

- **UART6 is polled, not interrupt-driven.** The `coralmicro` fork moves the M7 debug console off LPUART6 onto LPUART7 to free the UART6 header for the OBC link; since the vendor’s interrupt-driven driver still dispatches through per-instance state that is null for LPUART6 once the console moves, an LPUART6 interrupt would hard-fault, so the OBC link is polled I/O only.
- **SEQ survives reboots**, persisted to `/cloud_seq` after every capture, so a mid-flight reset does not collide with already-archived frames on the OBC’s SD card.

An unrecoverable initialisation failure triggers a self-reboot after a fixed delay (`FatalRestart`) rather than hanging – there is no one in orbit to press reset. Debug-only tooling (HTTP/RPC image preview, burst capture) is flagged in `docs/TODO.md` for removal before flight and does not ship on the UART data path.

2.3.4 Wire Interface

Every cycle the payload sends one image packet to the OBC over UART6 at 115200 baud, 8N1: the full 224×224 grayscale frame, the 2-byte cloud fraction, and the sequence number, CRC-16 protected. The OBC archives the frame to SD keyed by SEQ and forwards only the fraction record to the downlink. The byte-level framing is the authoritative UART Protocol Specification in [interface_docs/UART_PROTOCOL.md](#), not reproduced here.

2.4 Thermal Envelope

The payload carries its own requirement, distinct from the rest of the platform: the Coral’s interior must stay above 0°C throughout the flight (MR-009 scopes this floor to the payload specifically). Passive insulation alone cannot hold that floor through the night-cruise cold case; a small thermostatic survival heater ($\sim 2\text{--}3\text{ W}$) was evaluated as a mitigation. Its cost is quantified in the EPS chapter: at 2.5 W and 30% duty the platform’s power margin falls from $\sim 3.1\times$ to $\sim 1.9\times$ – still clear of the $1.5\times$ floor.

Given the limited time before flight, the team chose passive-only thermal management for this iteration and mitigated the risk by placement instead: every high-internal-heat component (boost converter, batteries, Coral) sits close together so the platform’s own dissipation stays concentrated. This is a managed risk – the heater remains budgeted as a fallback, and the freezer/vacuum session validates whether passive-only holds in practice.

Chapter 3

Command and Data Handling (CDH)

3.1 Responsibilities

3.2 Interfaces

3.3 Design Decisions

Chapter 4

Telemetry, Tracking and Command (TT&C)

4.1 Subsystem Overview

The Telemetry, Tracking and Command (TT&C) subsystem is the operational link between Lakitu and its ground crew. It periodically downlinks the current spacecraft state, provides the GNSS information used to track and recover the balloon payload, accepts a small set of uplink messages, and exposes radio health to Fault Detection, Isolation and Recovery (FDIR). The implemented link is bidirectional half-duplex LoRa using an RFM95W/SX1276-class modem at 868 MHz. The flight computer is an STM32L476RG and the ground radio is driven from a PC through a CH347 USB-to-SPI adapter.

The complete end-to-end system is shown in Figure 4.1. An important design boundary is that Flight Software (FSW) owns construction of the telemetry packet, while TT&C owns radio scheduling, delivery attempts, acknowledgement handling and modem health. FDIR, rather than TT&C, owns the policy that decides when a radio failure is severe enough to request recovery.

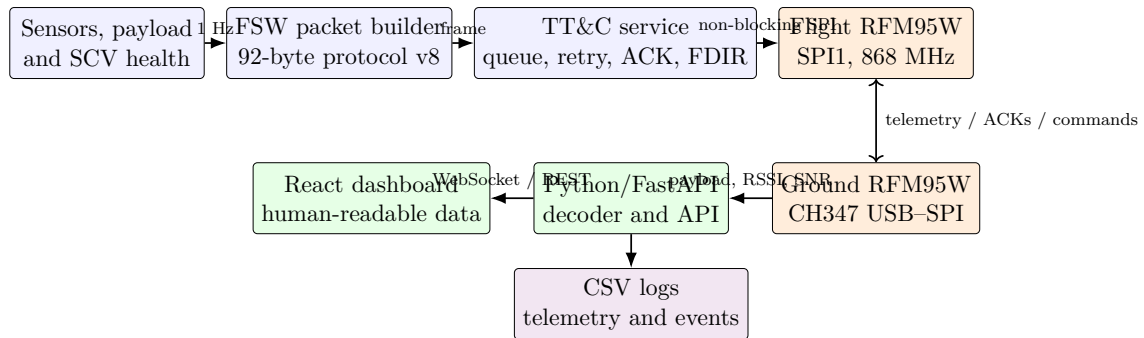


Figure 4.1: End-to-end TT&C architecture and ownership boundaries.

4.1.1 Responsibilities

The implemented subsystem configures and verifies the modem, maintains continuous receive outside transmissions, schedules a nominal frame every 20 s and on significant events, and transmits without blocking the spacecraft superloop. It retains an unacknowledged frame and retries the exact same bytes, sequence and CRC up to three total attempts.

It also parses acknowledgements and commands, rejects malformed or replayed traffic, exposes raw health to FDIR, and executes asynchronous isolation and recovery actions.

On the ground, TT&C receives and validates frames, restores engineering units, records link metadata and events, acknowledges valid downlinks, and provides live and historical data to the operator dashboard. Tracking is not a separate RF ranging function: it comes from the GNSS latitude, longitude, altitude, speed, course and time carried in every telemetry frame.

4.2 Requirements and Traceability

Table 4.1 maps the allocated requirements to the present implementation. “Implemented” means that behavior exists in the repository; it does not replace the environmental, range or compliance test named by the requirement.

Table 4.1: TT&C requirement traceability.

ID	Requirement intent	Implementation evidence	Verification state
MR-004	Telemetry to 40 km altitude / 125 km worst-case slant range	LoRa physical layer and a requirements-level link-budget assumption are defined.	Range test open
FR-022	Approximately one packet every 20 s	TTC_TELEMETRY_INTERVAL_MS = 20000; event packets are also supported.	Timing implemented; duty-cycle issue open
FR-023	Standard packet with state, position, IMU, payload, battery and integrity check	Protocol v8 carries all named data plus health; CRC-16 protects bytes 0–89.	Implemented and decoder-tested
FR-024	European 868 MHz ISM-band operation	Flight registers select 868.000 MHz.	In band; frequency alignment open
FR-025	Receive, decode and display human-readable telemetry	RFM95W receiver, decoder/store/API, WebSocket and React views are present.	Implemented; hardware demonstration open
FR-026	SF9, 125 kHz bandwidth, coding rate 4/5	Flight and ground configure explicit-header LoRa, SF9, BW125 and CR 4/5.	Registers implemented
PR-004	At least 10 dB margin at 125 km	Section 4.10 recomputes the conditional margin.	Range test open
IF-005	OBC-to-radio SPI no faster than 10 MHz	SPI1 is 625 kbit/s with interrupt-driven transfers.	Implemented
IF-007	FSW supplies phase and system state in a defined format	FSW_BuildTelemetryPacket() consumes datapool, SCV and TTC health.	Implemented

ID	Requirement intent	Implementation evidence	Verification state
CR-005	Power and channel-use constraints	Flight PA is +14 dBm; the 20 s/513 ms schedule does not meet the project's stated 1% arithmetic.	Duty-cycle closure open
SR-004	No interference with balloon flight-control systems	No interference evidence is stored in the repository.	Operator test open

4.3 Physical and Radio Interface

The RFM95W is connected to STM32 SPI1 in master mode. Transfers are full duplex, 8-bit and interrupt driven. Chip select and reset are ordinary GPIOs; the driver polls the modem IRQ register, so the implemented path does not need a DIO interrupt line.

Function	STM32 pin	Board label	RFM95W signal
SPI clock	PA5	D13	SCK
SPI MISO	PA6	D12	MISO
SPI MOSI	PA7	D11	MOSI
Chip select	PB6	D10	NSS / CS
Reset	PC7	D9	RESET

Table 4.2: Flight radio electrical interface.

The CubeMX configuration and hand-maintained initialization both set SPI1 to 625 kbit/s. The driver verifies the RFM95W version register (expected 0x12) and reads back frequency, modem and sync-word registers. A transport failure produces `INIT_FAIL`; a readable but unexpected value produces `CONFIG_FAIL`.

Parameter	Flight value	Ground implementation
Carrier frequency	868.000 MHz (FRF D9 00 00)	Backend default 868.100 MHz; must be aligned
Spreading factor / bandwidth	SF9 / 125 kHz	SF9 / 125 kHz
Coding rate / header	4/5 / explicit	4/5 / explicit
Radio payload CRC / preamble	Enabled / 8 symbols	Enabled / 8 symbols
Sync word	0x12	0x12
Transmit power	+14 dBm (PA boost)	+10 dBm default
Receive mode	Continuous between TX	Continuous except during TX
Telemetry payload	92 bytes	Accepts exactly 92 bytes

Table 4.3: LoRa configuration derived from flight and ground source code.

The 100 kHz center-frequency difference is a current integration defect. Both values are inside the intended band, but transmitter and receiver should use the same value. A shared configuration or explicit pre-flight register/readback check is recommended.

4.4 Telemetry Generation and Scheduling

The superloop calls `TTC_Service()` on every iteration because TT&C and the LoRa driver are cooperative, non-blocking state machines. They advance at most a small SPI or state step per call, allowing FDIR and the independent watchdog to continue while a radio operation is in progress.

When `TTC_TelemetryDue()` is true, FSW builds a packet from the 1 Hz sensor datapool, persistent Spacecraft Vector (SCV), and volatile TT&C health/command snapshots. A frame is due before the first transmission, after 20 s from the last successful TX, or when an immediate request is latched. Flight-phase changes and received commands set this latch. If a previous frame still awaits its ACK, no new frame is built: the pending sequence is completed or abandoned first.

Current sensor validity is separate from the numeric value. If a source becomes invalid, its last known value is retained but its validity bit is cleared. Before the first valid observation, signed fields use the minimum integer and unsigned fields use their maximum. Ground must therefore interpret validity, the FDIR fault mask and the equipment-enabled mask together.

4.5 Protocol-v8 Telemetry Frame

The downlink is a packed, little-endian, fixed-width 92-byte binary frame. Engineering values are scaled and clamped to integer units; ground reverses the scaling. Figure 4.2 shows the allocation and Table 4.4 gives the complete definition.

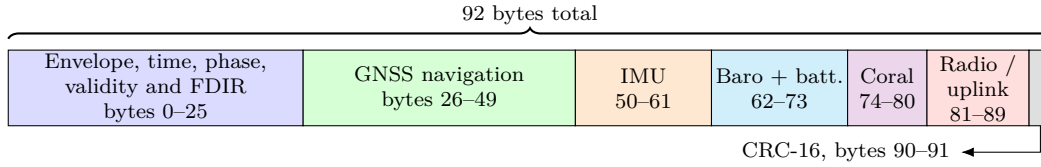


Figure 4.2: Protocol-v8 byte allocation by information group.

Table 4.4: Complete protocol-v8 telemetry wire layout.

Offset	Bytes	Wire field	Meaning and scale
0	1	packet_type	0x01 identifies telemetry
1	1	protocol_version	0x08; ground accepts v8 only
2	2	sequence_number	Unsigned sequence; unchanged on retry
4	4	tx_uptime_s	OBC uptime in seconds
8	4	mission_elapsed_s	Persistent mission time in seconds
12	2	boot_count_sat	Boot count, saturated at 65,535
14	1	flight_phase	Six-state flight-phase enum
15	1	reset_reason	Unknown, power-on, watchdog or software reset
16	1	validity_flags	Bits 0–4: GNSS, IMU, baro, battery, Coral
17	1	i2c_bus_state	0 idle, 1 restart, 2 hold
18	2	equipment_enabled	Policy mask for enabled equipment
20	2	equipment_faults	Debounced FDIR fault mask
22	2	sample_age_ms_sat	Packet tick minus datapool tick; 65,535 unknown/overflow
24	1	watchdog_reset_count	Persisted watchdog-reset count
25	1	sd_fault_count	Persisted SD fault count
26	4	latitude_e7	Latitude, degrees $\times 10^7$
30	4	longitude_e7	Longitude, degrees $\times 10^7$
34	4	gnss_altitude_dm	GNSS altitude in decimetres
38	2	vertical_speed_cms	cm/s, positive upward
40	2	ground_speed_cms	Horizontal speed in cm/s
42	2	course_cdeg	Course in centidegrees, 0–35,999
44	4	gnss_utc_sod	UTC seconds of day
48	1	gnss_satellites	Satellites used in solution
49	1	gnss_fix_type	GNSS fix-type enum
50	2	accel_x_mg	X acceleration in milli-g
52	2	accel_y_mg	Y acceleration in milli-g
54	2	accel_z_mg	Z acceleration in milli-g

Offset	Bytes	Wire field	Meaning and scale
56	2	gyro_x_ddeg_s	X angular rate in 0.1 degrees/s
58	2	gyro_y_ddeg_s	Y angular rate in 0.1 degrees/s
60	2	gyro_z_ddeg_s	Z angular rate in 0.1 degrees/s
62	4	baro_pressure_pa	Barometric pressure in pascals
66	4	baro_altitude_dm	Ground-relative altitude in decimetres
70	2	baro_temperature_cdeg	Temperature in 0.01 degrees C
72	2	battery_mv	Battery voltage in millivolts
74	2	coral_sequence_low	Low 16 bits of payload result sequence
76	2	coral_fraction_q16	Cloud fraction, 0–65,535 maps to 0–100%
78	1	coral_status	Coral status bit mask
79	2	coral_age_s_sat	Coral result age in seconds; saturated
81	1	lora_last_event	Latest modem event enum
82	1	lora_consecutive_failures	Consecutive radio failures
83	1	lora_recovery_count	Recovery attempts since boot
84	1	lora_rx_state	Bits 0–2 RX status; bit 3 continuous RX active
85	1	lora_tx_fault_count_sat	Lifetime failed TX attempts, saturated
86	1	lora_ack_timeout_count_sat	Exhausted ACK budgets, saturated
87	2	last_command_id	Last command ID observed by flight
89	1	uplink_state	Bits 0–2 command status; 3–5 ACK status
90	2	crc16	CRC-16/CCITT-FALSE over bytes 0–89

The CRC uses polynomial 0x1021 and initial value 0xFFFF. Ground first rejects any payload not exactly 92 bytes, then checks packet type, protocol version and application CRC. The RFM95W packet CRC is an additional physical-link check. Sequence numbers identify gaps, duplicates and a new session after reboot.

Protocol v8 reduced the former 155-byte raw-structure frame to 92 bytes, a 41% payload reduction. Estimated airtime fell from about 800 ms to 513 ms, about 36% per attempt. Data without an onboard producer, such as GNSS dilution of precision and MCU/IMU temperature, deliberately receive no wire bytes.

4.6 Reliable Downlink and Uplink Protocol

LoRa supplies the radio packet, while reliability is implemented at the application layer. Figure 4.3 summarizes a transaction.

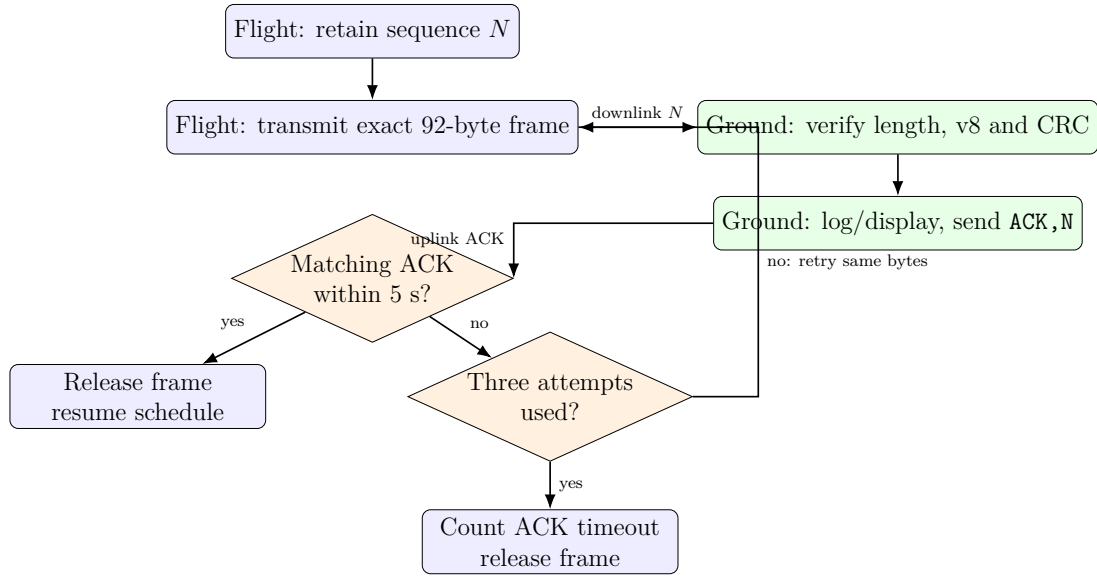


Figure 4.3: Application-level acknowledgement and retry sequence.

After *TxDone*, flight returns to continuous RX and waits 5 s for **ACK, <sequence>**. A match releases the frame; the last accepted ACK is classified as duplicate; another valid sequence is unexpected. ACKs require no ACK-of-ACK. Ground sends the ACK only after full application validation and acknowledges a duplicated sequence again. A duplicate normally means downlink succeeded but the earlier ground ACK did not reach flight in time.

4.6.1 Commands and Replay Protection

Flight accepts ASCII uplinks up to 64 bytes. The implemented classes are:

Wire form	Meaning	Flight response
ACK, <sequence>	Acknowledge telemetry	Update independent ACK-status latch
CMD, <id>, REQ_TELEMETRY	Request out-of-cycle telemetry	Record ID/status and schedule immediate telemetry
CMD, <id>, <other verb>	Valid but unsupported command	Latch UNSUPPORTED and schedule telemetry
Malformed CMD or ACK	Invalid format	Latch INVALID_FORMAT and schedule telemetry

Table 4.5: Implemented uplink message classes.

Command IDs are non-zero 16-bit values. Flight maintains a 16-command sliding replay window with wrap-safe arithmetic: a command is accepted once and a repeated ID is reported as duplicate. Ground serializes command transactions, assigns a stable ID, and by default retries three times with 5 s TX and 5 s flight-response timeouts. Confirmation is the command ID and **ACCEPTED**/**DUPLICATE** status echoed in valid telemetry. Command and telemetry-ACK status are independent latches, so an ACK cannot erase a command confirmation. **REQ_TELEMETRY** is currently the only operational verb.

4.7 Flight Software State Machines

The TT&C layer and LoRa driver are asynchronous. Initialization, send, receive-start and recovery calls queue work, and repeated service calls advance it. This prevents radio waits from starving the watchdog or other subsystems.

State family	Purpose
Startup initialization / RX	Reset/configure, verify registers, enter continuous RX
Idle	Poll uplink, reconcile faults, start telemetry/retries and FDIR actions
TX wait / RX restart	Await <i>TxDone</i> /timeout, record result, restore RX
Action isolation	Quiesce SPI and hold modem in reset
Action initialization / RX	Full recovery or return-to-service, then restore RX

Table 4.6: High-level TT&C state-machine responsibilities.

Driver transfers use `HAL_SPI_TransmitReceive_IT()`; modem IRQ flags are polled for *RxDone*, CRC error and *TxDone*. SPI timeouts trigger a bounded interrupt abort, and the driver can reinitialize SPI if it does not quiesce. Oversized observations are consumed and returned as length errors so stale FIFO data cannot poison the next packet.

4.8 FDIR and Radio Health

TT&C records facts; FDIR applies policy. The raw snapshot contains readiness, continuous-RX state, consecutive/lifetime failures, successful TX/RX times, recovery activity, isolation and exhausted-ACK count. FDIR reconstructs a pass/fail history from the counters.

Condition / action	Implemented policy	Ownership
Recovery request	5 consecutive failures, or 12 failures in last 20 attempts	FDIR decides; TT&C executes
Recovery cooldown	At least 10 s between requests	FDIR
SCV LoRa fault	Set at 9 failures in last 10; clears as window improves	FDIR
Full recovery	Reset, configure, read back, restart RX	TT&C / driver
RX restart	Re-enter RX without full reset if otherwise ready	TT&C / driver
Isolation	Preempt activity and hold reset	API exists; automatic give-up policy open
Return to service	Reinitialize isolated/faulted modem	TT&C / driver

Table 4.7: Implemented LoRa FDIR policy and ownership.

FDIR sets `EQUIPMENT_LORA` in a shared reinit bitmask. `TTC_Service()` consumes it and advances the action later. Only one action may be pending, except isolation can preempt. Requests are idempotent. Telemetry reports the latest event, failures, recoveries, RX state, lifetime TX faults and ACK timeouts. Ground combines this snapshot with end-to-end evidence such as a received frame, RSSI/SNR, duplicates and command outcomes.

4.9 Ground Segment

The ground segment has three layers. Python controls an RFM95W through CH347, reads payload/RSSI/SNR/CRC, and restores continuous RX after transmissions. FastAPI validates and decodes v8, tracks lost/duplicate sequences, keeps bounded history, writes telemetry and event CSVs, exposes REST, and broadcasts over WebSocket. The React dashboard provides Overview, Health, Position/Flight Phase, Link Analytics, Commands and Raw Telemetry views.

Invalid and physical-CRC observations remain available for diagnosis but never replace the latest valid spacecraft state. Main interfaces are `/api/status`, `/api/latest`, `/api/history`, `/api/stats`, `/api/uplink-log`, `/api/send-ascii` and `/ws/telemetry`. The backend can run without radio for UI/API work. User verbs are limited to 48 ASCII characters so the envelope fits flight RX.

4.10 Link Budget, Airtime and Channel Use

The requirements use 125 km as worst-case slant range. Free-space path loss is

$$L_{\text{FS}} = 32.44 + 20 \log_{10}(d_{\text{km}}) + 20 \log_{10}(f_{\text{MHz}}), \quad (4.1)$$

which gives 133.2 dB at 125 km and 868 MHz. The requirement assumes -137 dBm receiver sensitivity at SF9/BW125. Antenna gains, cable, polarization and installation losses, and measured sensitivity are absent, so Table 4.8 is conditional rather than RF qualification.

Term	Downlink	Uplink
Transmit power	+14.0 dBm	+10.0 dBm
Antenna gains (illustrative)	0.0 dBi	0.0 dBi
Free-space path loss	−133.2 dB	−133.2 dB
Other losses (illustrative)	−3.0 dB	−3.0 dB
Predicted receive power	−122.2 dBm	−126.2 dBm
Assumed sensitivity	−137.0 dBm	−137.0 dBm
Illustrative margin	14.8 dB	10.8 dB

Table 4.8: Conditional 125 km link budget using repository assumptions.

With no extra losses, margins are 17.8 dB downlink and 13.8 dB uplink. With the illustrative 3 dB allowance, uplink exceeds the 10 dB target by only 0.8 dB. The requirements file states 26.5 dB without a reproducible calculation; the zero-gain budget

above does not reproduce it. Acceptance requires measured sensitivity, characterized antennas/cables and representative range testing.

At SF9, BW125, CR 4/5, explicit header, CRC and eight-symbol preamble, symbol time is 4.096 ms and 92-byte airtime is about 513 ms. At a strict 20 s period:

$$\frac{0.513}{20} \times 100\% \approx 2.57\%. \quad (4.2)$$

This conflicts with the 1% channel-use assumption in FR-022/CR-005 before event packets or retries. At the same modulation, one 513 ms transmission needs at least a 51.3 s interval to average 1%. The project must close this by a permitted channel/access regime, schedule/modulation change, or another reviewed compliance basis. Retries and ground ACKs belong in the final analysis.

4.11 Verification Evidence

Repository tests provide the following host-side evidence:

- v8 decoder tests cover size, scaling, sentinels, CRC and legacy rejection;
- TT&C harnesses cover startup recovery, RX faults, idempotency, isolation, retries, command/ACK latches and the FDIR request bit;
- driver harnesses cover oversized RX, 255-byte FIFO transfer, abort rejection/timeout recovery and abort callbacks;
- source-contract tests enforce interrupt-driven SPI, non-blocking design, FDIR policy ownership and queued recovery interfaces.

Remaining acceptance work is a two-radio register dump; end-to-end frame, ACK and command demonstration; five-minute schedule measurement; injected CRC, SPI, ACK-loss and reset faults; sensitivity and antenna characterization; range test; actual channel-occupancy log including retries; and an interference check coordinated with the balloon operator.

4.12 Design Decisions and Known Limitations

The central decisions are a portable fixed-integer frame, continuous receive, reliability and replay protection above LoRa, non-blocking state machines, independent command/ACK latches, and FDIR ownership of recovery policy. These improve watchdog safety, diagnostics and operational observability.

Open limitations are:

- flight and ground center frequencies differ by 100 kHz;
- 20 s scheduling and 513 ms airtime conflict with the stated 1% target;
- range margin depends on unverified sensitivity and unspecified antennas;
- REQ_TELEMETRY is the only implemented command verb;
- automatic give-up escalation to long-term isolation remains open;
- operator/deployment procedures need a dedicated ground-station runbook;
- no artifact yet proves 125 km range, regulatory closure or non-interference.

Closing these items is primarily integration and verification work; the packet, reliable transactions, decoder and core radio/FDIR state machines exist.

Chapter 5

On-Board Computer (OBC)

5.1 Responsibilities

The OBC is the single flight computer of the platform: one microcontroller hosts all subsystem software – CDH sensor acquisition, the FSW state machine, FDIR monitoring, TT&C packetization, and SD-card logging – with the Coral payload as the only other processor on board. Its responsibilities are:

- **Compute platform.** Execute all flight software in a fixed 1 Hz major cycle (CDH_Update → FSW_Update → FDIR_Update → SD logging → TT&C service).
- **Timekeeping.** Provide the mission time base (`HAL_GetTick()`); the Coral has no RTC, so the OBC timestamps every payload packet.
- **Mass storage.** Archive housekeeping telemetry and payload image frames to the SD card (FR-016/FR-017, Section 5.3.1).
- **Platform health.** Run the independent watchdog (FR-011), classify reset causes into the SCV, and digitize battery voltage from the EPS divider (IF-008).

5.2 Interfaces

Every peripheral of the spacecraft terminates at the OBC (Figure 5.1). Table 5.1 lists the bus allocation as configured in the final STM32CubeMX pinout.

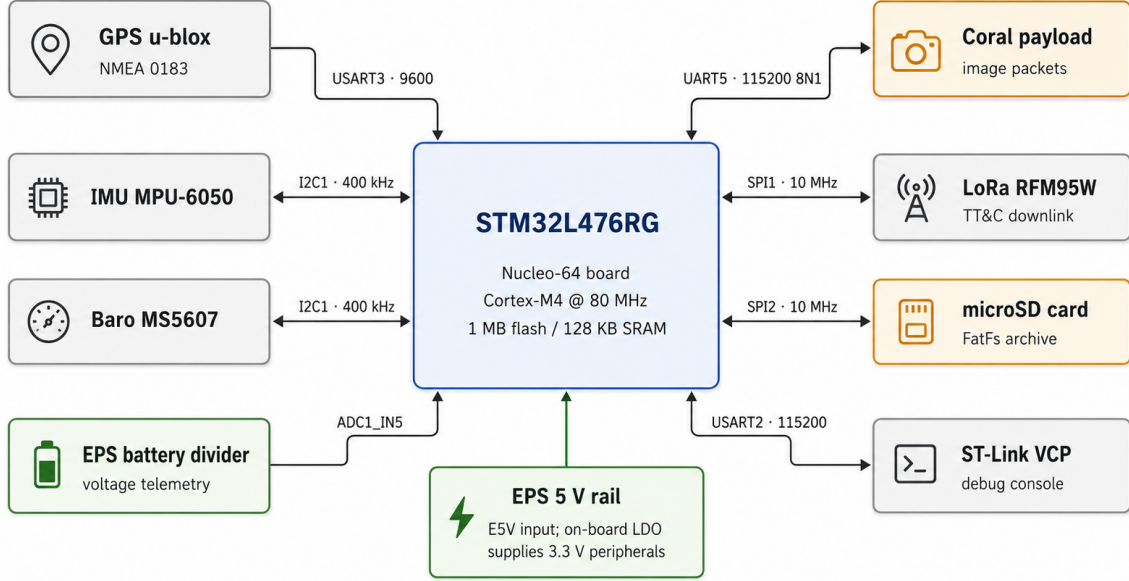


Figure 5.1: OBC bus architecture.

Bus	Pins	Peripheral	Configuration	Owner
USART3	PB10/PB11	GPS (u-blox)	9600 baud, NMEA 0183	CDH
I2C1	PB8/PB9	IMU MPU-6050, MS5607	400 kHz, shared bus	CDH
UART5	PC12/PD2	Coral payload	115200 8N1, binary + CRC-16	OBC/CDH
SPI1	PA5–PA7	LoRa RFM95W	10 MHz	TT&C
SPI2	PB13–PB15	SD card	312.5 kHz init, 10 MHz run	OBC
ADC1	PA0 (IN5)	Battery divider (IF-008)	3.2× firmware scale	CDH
USART2	PA2/PA3	ST-Link virtual COM	115200, debug console	OBC

Table 5.1: OBC bus and pin allocation

Power comes from the EPS 5 V rail into the Nucleo’s E5V input; the board’s on-board regulator derives the 3.3 V rail that supplies the sensors, SD card, and LoRa module (see EPS chapter).

5.3 Design Decisions

Hardware selection. The OBC is a Nucleo-64 board with an STM32L476RG: Cortex-M4 at 80 MHz, 1 MB flash, 128 KB SRAM, from ST’s ultra-low-power line. It was chosen over higher-performance parts because the 1 Hz control problem does not need more compute, and the low-power core keeps the OBC’s share of the EPS budget small (~ 0.4 W including sensors and LoRa). The Nucleo format adds an on-board ST-Link debugger/programmer and breakout headers that suit the hand-wired integration. The choice imposes two constraints that shaped the software: a 50 KB payload frame occupies 39% of SRAM, so

images can never be buffered whole; and the Nucleo-64 has no SD slot or SDMMC wiring, so the SD card runs over SPI.

Superloop instead of an RTOS. All software runs in a single bare-metal loop with a fixed call order and no preemption. With one 1 Hz rate group and no hard concurrent deadlines, an RTOS would add stack-per-task RAM cost and priority/locking failure modes without buying anything; the superloop makes timing trivially auditable. The independent watchdog (IWDG) is refreshed exactly once per loop iteration with a timeout below the 10 s FR-011 limit, so any hang anywhere in the cycle ends in a clean reset rather than a silent loss of the mission.

Boot order and degraded boot. FDIR initializes first, before any peripheral driver, so reset classification and the watchdog-reset counter run even if later initialization fails. Each subsystem init is gated on its SCV equipment flag: a peripheral that fails to start is flagged faulty and the boot continues – the OBC always comes up, in a degraded mode if necessary, rather than hanging in an init retry loop. The boot-time battery-voltage guard (EPS chapter) is checked before normal operation resumes.

5.3.1 SD Card Storage

The SD card holds the two on-board data products: a 1 Hz housekeeping log (FR-016) and the payload image archive, keyed by the payload sequence number **SEQ** so each downlinked cloud fraction can be matched to its source frame post-flight. The stack is a bare-metal SD-SPI driver under FatFs (Figure 5.2): card initialization is clocked at 312.5 kHz to respect the sub-400 kHz SD specification window, then raised to 10 MHz for operation.

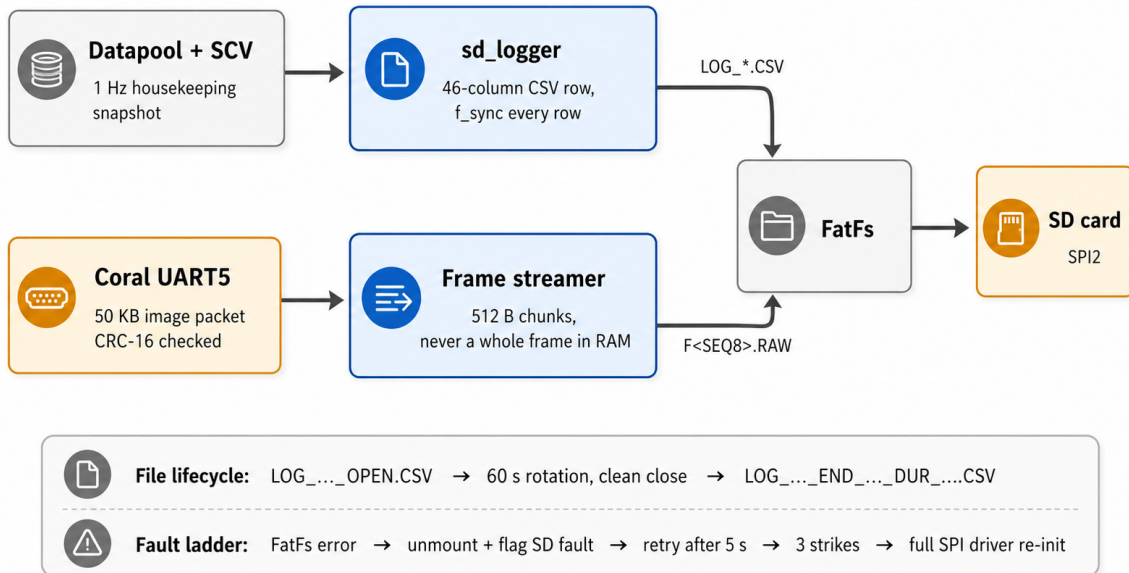


Figure 5.2: SD storage pipeline, file lifecycle, and fault ladder.

Housekeeping log. Each 1 Hz cycle appends one 46-column CSV row: the full datapool snapshot (GPS, IMU, barometer, battery, the 16-byte Coral status block) plus the complete SCV, all scaled to integers on write so the log needs no floating-point

formatting and parses unambiguously. Three choices bound the damage any single failure can do:

- **One file per minute.** The log rotates every 60 s, so a corrupted file costs at most one minute of data.
- **Sync every row.** `f_sync` runs after each record; power loss loses at most the row being written.
- **Self-identifying files.** A file is created as `LOG_<session>_START_<t>_OPEN.CSV` and renamed on clean close to carry its end time and duration – any file still named `_OPEN` after the flight is known to have died mid-write. Session numbers are recovered by directory scan, so numbering survives reboots without persistent state.

Image archive. Payload frames arrive on UART5 as 50 196-byte CRC-16-protected packets and are streamed to one file per frame (`F<SEQ8>.RAW`) through a 512-byte chunk buffer – at no point is a whole frame held in RAM, which the 128 KB SRAM budget rules out. A failed CRC or a write error drops that frame only; the next `SEQ` starts a fresh file.

Fault handling. Any FatFs error closes the file, unmounts the volume, sets the SD bit in `equipment_faults`, and schedules a retry 5 s later; after three consecutive faults the SPI driver is fully re-initialized from scratch. The card is declared healthy again only after a completed write-and-sync, not merely a successful mount. The entire SD path is deliberately fail-soft: telemetry reaches the ground over LoRa regardless of card state, so losing the SD card loses the image training archive, never the science result or the flight.

Chapter 6

Electrical Power System (EPS)

6.1 Design Overview

The EPS delivers a single regulated 5 V rail to the platform from a two-cell parallel Li-ion pack, with fault protection and battery-voltage telemetry to the OBC (IF-008). It is sized against MR-002 (six-hour flight), PR-008 ($\geq 1.5\times$ power margin), IF-009 (5 V rail behaviour under load), SR-001/002/003 (over-current, reverse-polarity, over-discharge protection), and CR-001 (mass).

The pack is $2\times$ Panasonic NCR18650B (unprotected, parallel, 2×3400 mAh, 25.1 Wh nominal), chosen for energy density with well-characterised cold performance (-20°C rated, so the battery is not the platform’s thermal bottleneck) and a hermetically sealed can that tolerates the ~ 1 kPa stratospheric environment without LiPo’s pouch-swell risk. A USB power bank, LiPo pouch, and lithium-primary cells were all rejected on cold performance, altitude-pressure risk, and non-rechargeability respectively. Because the cells are unprotected, over-current, reverse-polarity, and over-discharge protection is entirely external (Section 6.3). A boost-only (not buck-boost) topology is used because the battery voltage (3.0–4.2 V) is always below the 5 V rail target.

6.2 Power Budget

The initial budget overestimated the payload: it modelled the Edge TPU at full inference power (1.8 W) for 40% of the flight. A review corrected this to the datasheet active-power figure (~ 3 W) at a realistic duty – Edge TPU inference latency is millisecond-scale against a 1 Hz capture cycle, so the TPU is only briefly active per second – and removed a duplicated boost-efficiency term from the battery-referred conversion.

Quantity	Before	After
Total average battery power	~ 2219 mW	~ 1216 mW
Energy required (6 h)	13.3 Wh	~ 7.3 Wh
Usable battery (0.9 cold derate)	22.6 Wh	22.6 Wh
Margin	$1.70\times$	$\sim 3.1\times$

Table 6.1: Power budget before and after the review correction

The corrected budget, referred to the battery: Coral ~ 0.76 W (measured 158–180 mA idle; inference adds ~ 0.3 A for 2–2.4 ms, negligible in the 1 Hz average), Nucleo + sensors + LoRa ~ 0.40 W, regulator quiescent and series losses ~ 0.03 W – **~ 1.2 W total**. Over MR-002’s six-hour flight this is ~ 7.3 Wh against ~ 22.6 Wh usable, a $\sim 3.1\times$ margin against the PR-008 floor of $1.5\times$.

6.3 Power Architecture

The battery’s 2P pack feeds a 4 A polyfuse (FUZETEC FRG40016F), an SI2301 P-MOSFET reverse-polarity stage, and a Pololu U3V40F5 5 V boost regulator. The regulator output feeds the Coral payload (gated by a GPIO-switched high-side P-MOSFET) and the Nucleo’s E5V input; the Nucleo derives 3.3 V on-board for the sensors, SD card, and LoRa module. A 22 k Ω -top voltage divider taps the post-fuse battery line to an OBC ADC input (IF-008) at a firmware multiplier of 3.2.

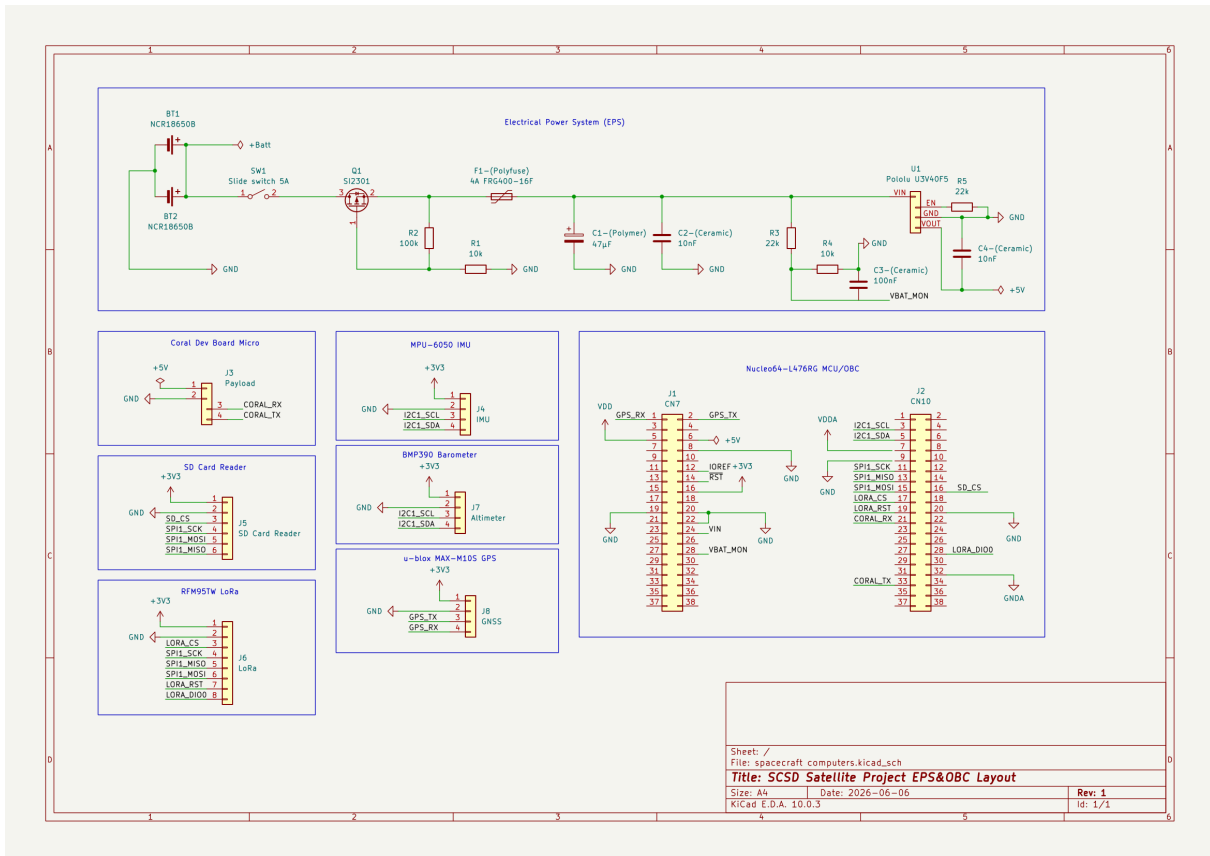


Figure 6.1: EPS and OBC schematic (KiCad).

Protection is layered: the 4 A polyfuse handles over-current (SR-001); the SI2301 stage is the sole reverse-polarity protection (SR-002), the regulator itself provides none; over-discharge (SR-003) is layered firmware-first (orderly shutdown at ~ 3.0 V/cell via IF-008 telemetry – close the SD log, send a final frame, halt) with a hardware EN-pin UVLO backstop below that. One interaction between the two layers was found and closed: the UVLO recovery point sits above the firmware shutdown threshold, so at end of discharge the relaxing open-circuit cell voltage could restart the converter and reboot the OBC into a loop. The fix is a boot-time guard – read the battery voltage first and

stay halted if it is still below the recovery point – making the hardware backstop alone sufficient to prevent over-discharge regardless of firmware state.

Payload power control uses the same GPIO-switched P-MOSFET as true cutoff (the RT1176 cannot software-cut its own supply), paired with a UART **SHUTDOWN** command so the payload quiesces before the rail is cut. A bulk capacitor originally planned at the Coral connector was dropped after bench measurement showed the real inference transient to be roughly five times gentler than the worst-case assumption the budget had carried (Section 6.4); the regulator’s own capacitance absorbs the load step without help.

The platform stacks as a 10×10×15 cm 1U volume: TT&C and CDH on top, batteries, EPS, OBC, and the Coral payload below. The EPS is a hand-wired perfboard carrying the battery input, polyfuse, reverse-polarity MOSFET, boost regulator, and telemetry divider, with header rows broken out for the Coral power connector and the OBC power/telemetry lines.

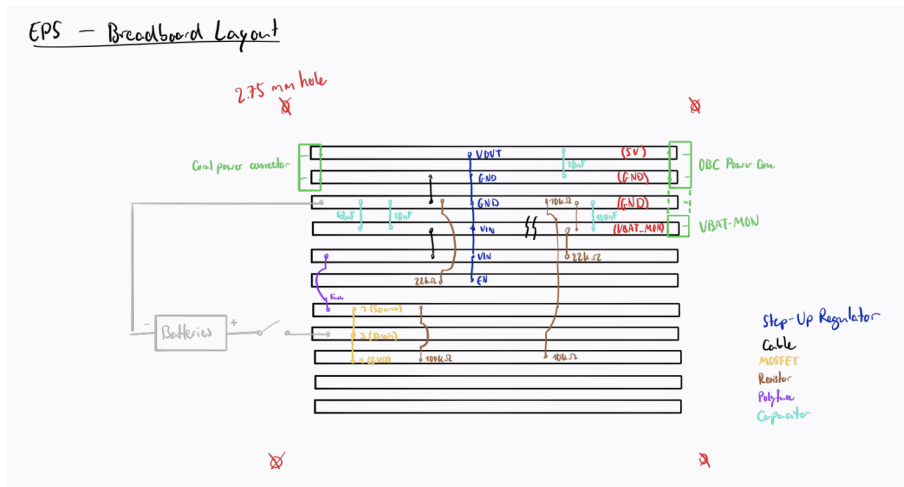


Figure 6.2: EPS perfboard layout, as built.

6.4 Verification

The EPS was bench-tested 2026-07-07 against a bring-up ladder: visual inspection, continuity, current-limited power-up, battery power-up, increasingly realistic loads, ending with the Coral integration test. The rail regulates at a constant 5.06 V across the full 3.0–4.2 V battery range; the telemetry divider is exact; the SI2301 stage fully blocks a reversed input; and the measured inference transient (~ 0.3 A above baseline, 2.0–2.4 ms, self-contained) is far gentler than the multi-amp spike the original budget assumed.

Req.	Requirement	Verification
MR-002	Operate 6 h without external power	Analysis: budgeted demand (7.3 Wh) is a third of usable capacity (22.6 Wh); steady 5.06 V regulation and no throttling observed through bench testing.
PR-008	Margin $\geq 1.5\times$ at 6 h, 0.9 cold derate	$\sim 3.1\times$ shown by analysis, comfortable even under the conservative derate assumption.
IF-008	Battery telemetry within 5%	Divider ratio measured exact ($3.2\times$ at 3.7 V); combined with the STM32 ADC's sub-1% typical error, the chain sits well inside the 5% budget.
IF-009	5 V rail ≥ 4.5 V under 2 A peak	Static 5.06 V across the full battery range; ~ 30 mV dip against a 160 mV budget ($\sim 5\times$ margin) at the Coral connector during inference, confirmed on two independent setups.
SR-001	Over-current protection	4 A hold test passed (5 min, no trip); the FRG40016F's ~ 8 A trip point is a manufacturer-guaranteed datasheet spec, so hold test plus trip-curve analysis is sufficient closure.
SR-002	Reverse-polarity protection	Reversed input draws zero current, rail stays at 0 V.
SR-003	Over-discharge protection with hysteresis	Hardware UVLO backstop (2.8 V cut, 3.1 V recover) is independently sufficient regardless of firmware state; firmware adds an earlier orderly shutdown, with the boot-time guard (Section 6.3) closing the one threshold interaction found.
PR-006	Boot ≤ 30 s (EPS share ≤ 2 s)	Regulator soft-start and the clean, hiccup-free power-up observed throughout bench testing give large margin against the 2 s allocation.
MR-007 MR-009	/ Subsystems within rated ranges	EPS scope verified: the battery is rated to -20°C , well clear of any platform requirement, so it is not a thermal bottleneck. The payload's separate 0°C floor is its own subsystem's concern, covered in the Machine-Learning Payload chapter.
MR-008 CR-001	/ Flyable mass ≤ 650 g	EPS assembly (board + pack) measured 15.09 g, well inside its 200 g sub-budget.

Table 6.2: EPS requirement verification summary

Chapter 7

Flight Software

7.1 Mission Phases

The mission is divided into six flight phases, each representing a physically distinct regime of the stratospheric balloon flight.

Phase	Physical description
Standby	System powered on, sitting on ground. All sensors initialising. Ground pressure baseline (P_ground) recorded here as reference for all altitude calculations.
Launch	Balloon released. Sharp upward jerk, brief chaotic motion lasting seconds. Transition from ground-constrained to free flight.
Ascent	Balloon stabilised, climbing at 3-6 m/s steadily. Longest active phase, typically 1.5-2 hours. Temperature drops significantly with altitude.
Cruise	Balloon fully expanded at float altitude (25-35 km). Vertical velocity near zero, altitude oscillating within a bounded range. Duration 30 min to several hours.
Descent	Balloon envelope bursts. Rapid fall begins, IMU sees sudden shift from near 0g float to 1g. Parachute deploys, stabilising descent rate.
Landing	System touches down. All motion stops. Terminal state, no further FSM transitions. Logging and telemetry continue for recovery; shutdown timer starts.

Table 7.1: Mission phases and physical description

Chapter 8

Structures

8.1 Responsibilities

8.2 Interfaces

8.3 Design Decisions